

Proyecto Interdisciplinar

Simulador de Energías Renovables

Acceso a datos AED



Manual de Desarrollador

Autores:

LINGCHENG, JIANG

NATALY ALEXANDRA, IPIALES BARRIGAS

Índice

INTRODUCCIÓN.....	3
Identificación de entidades y atributos descriptivos.....	3
1. Placas Fotovoltaicas (Generadores).....	3
2. Elemento de Consumo.....	3
3. Proyecto.....	4
4. Usuario.....	4
Avances durante el desarrollo.....	5
Modelo Relacional.....	6
E/R.....	7
Diagrama UML.....	8
Configuraciones iniciales para el proyecto.....	9
Guía De Primer Contacto.....	10
Flujo Administrador.....	10
Flujo Usuario.....	10
Pruebas.....	11
Despliegue del Backend.....	18
Despliegue Web.....	18
Despliegue Backend.....	18
Conclusiones.....	19

INTRODUCCIÓN

El backend de este proyecto tiene como objetivo proporcionar la lógica y el soporte necesario para un simulador educativo de sistemas de energía solar para el módulo de **Gestión del montaje de instalaciones solares fotovoltaicas** del **Técnico Superior en Energías Renovables**.

El backend se encarga de gestionar la estructura de datos, las simulaciones de generación y consumo de energía, y el almacenamiento de proyectos y configuraciones de usuario.

La aplicación va a ser usada por usuarios, que tendrán que iniciar sesión para poder acceder a la plataforma de simulación. Cada usuario puede crear diferentes proyectos cuando está logueado, dichos proyectos tendrán un id, fecha de creación, nombre, y además dará información de si el elemento de consumo cubre las necesidades energéticas o no. Por otra parte, también hará un cálculo para avisar al usuario de que cantidad de energía es necesaria para que se cubran las necesidades energéticas.

Cada proyecto puede tener una lista de elementos, tanto de consumo como elementos que generan energía, en este caso paneles solares. Los elementos guardan información general como el id, nombre y coordenadas en la pizarra de creación. Además, los elementos de clase panel guardan información adicional como la marca, la eficiencia energética y la energía que son capaces de generar a lo largo de un día.

Por último, los elementos de consumo, además de guardar la información general de elementos, también almacenan información del consumo de energía que hacen en un día.

Identificación de entidades y atributos descriptivos

1. Placas Fotovoltaicas (Generadores)

Refleja la entidad placa, donde cada uno de las instancias de las placas que se generen/almacenen tendrán su información individual asociada con su identificador así como su marca, nombre, potencia nominal (Wp) y eficiencia.

Campos:

- id_placa: identificador único de la placa (clave primaria).
- marca: fabricante o marca de la placa. Tipo texto
- nombre: modelo o referencia comercial. Tipo texto
- potencia_wp: potencia máxima en vatios pico. Tipo Float
- eficiencia: rendimiento de conversión energética del panel. Tipo texto
- coordenadas: lugar que ocupa en la pizarra de trabajo

2. Elemento de Consumo

En esta tabla se almacena la información correspondiente a los elementos de consumo que van a estar relacionados con las placas. Cada elemento tendrá su identificador, nombre y consumo medio en vatios.

Campos:

- id_consumo: identificador único del elemento de consumo (clave primaria). Tipo int
- nombre_instalacion: nombre o descripción de la instalación que utiliza la energía. Tipo texto
- consumo_wt: cantidad de vatios que consume el elemento. Tipo Float
- coordenadas: lugar que ocupa en la pizarra de trabajo

3. Proyecto

Un proyecto representa las relaciones que se establecen entre placas fotovoltaicas y los elementos de consumo. Es el usuario quien las crea y las administra. Contiene información general sobre el proyecto, su energía necesaria y faltante, así como el usuario responsable.

Campos:

- id_proyecto: identificador único del proyecto (clave primaria). Tipo int
- nombre: nombre del proyecto. Tipo String
- fecha_creacion: fecha de registro del proyecto. Tipo LocalDateTime
- cubre: devuelve un booleano para saber si la energía es suficiente o no. tipo Boolean
- energíaNecesaria: cantidad de energía requerida por el proyecto (en kWh o Wp). Tipo Float

4. Usuario

Contiene la información personal y de acceso de los usuarios registrados en el sistema. Cada usuario puede crear y administrar múltiples proyectos fotovoltaicos.

De momento no se implementan metodologías de seguridad como hashing de contraseñas. Se implementaran en la segunda evaluacion

Campos:

- id_usuario: identificador único del usuario (clave primaria). Tipo Long
- nombreCompleto: nombre y apellidos del usuario. Tipo String
- fecha_nacimiento: fecha de nacimiento. Tipo LocalDate
- email: correo de contacto y autenticación. Tipo String
- contraseña: credencial de acceso al sistema. Tipo String

Avances durante el desarrollo

Tras la normalización, y para evitar atributos repetidos se decide crear una interfaz:

Elemento, que será una interfaz que contenga atributos comunes como:

- id_elemento. Tipo Long
- nombre. Tipo String
- coordenadaX. Tipo Float
- coordenadaY. Tipo Float

Esta interfaz será el elemento padre de:

ElementoConsumo tiene como atributo diferencial:

- consumo_wt, que indica el consumo diario del mismo. Tipo Double

ElementoGenerador tiene que atributos añadidos:

- marca: marca de la placa. tipo String
- eficiencia. Tipo Double
- potencia_wp (potencia que genera una placa al día. Tipo Double

Durante la implementación, se descubre que es necesario crear una clase **ProjectElement**, para cumplir con la cardinalidad y normalización. O si no, habría duplicados de datos si se usa elementos iguales en el mismo proyecto. Por ello, este tendrá los siguientes atributos:

- id: long (su propio id)
- id_proyecto: long
- id_elemento: long
- cantidad: int (cantidad del elemento en el proyecto)

Por otra parte, también se da cuenta de que es necesario guardar la relación entre los elementos de los proyectos dentro del simulador.

Por lo que también necesitaremos crear las entidades ProjectNode y ProjectNodeConnection. Siendo:

ProjectNode:

- posX: float (la posición para mostrar en pantalla)
- posY: float
- elementId: (referencia para obtener datos)

ProjectNodeConnection:

- nodeId1: long (guarda las conexiones entre los nodos del proyecto)

-
- nodeId2: long
 - connType: string

Modelo Relacional

resumiendo un poco los requisitos planteados en los apartados anteriores nos quedariamos con el siguiente modelo relacional:

user(userId, fullName, dateOfBirth, email, passwordHash, isAdmin)

project(projectId, name, updatedAt, energyEnough, energyNeeded, userId*)

projectNode(nodeId, units, positionX, positionY, elementIdReference, projectId*)

projectNodeConnection(connectionId, connectionType, projectId*, sourceNodeId*, targetNodeId*)

element(elementId, name)

generatorElement(elementId*, brand, efficiency, powerWatt)

consumerElement(elementId*, powerConsumption)

E/R

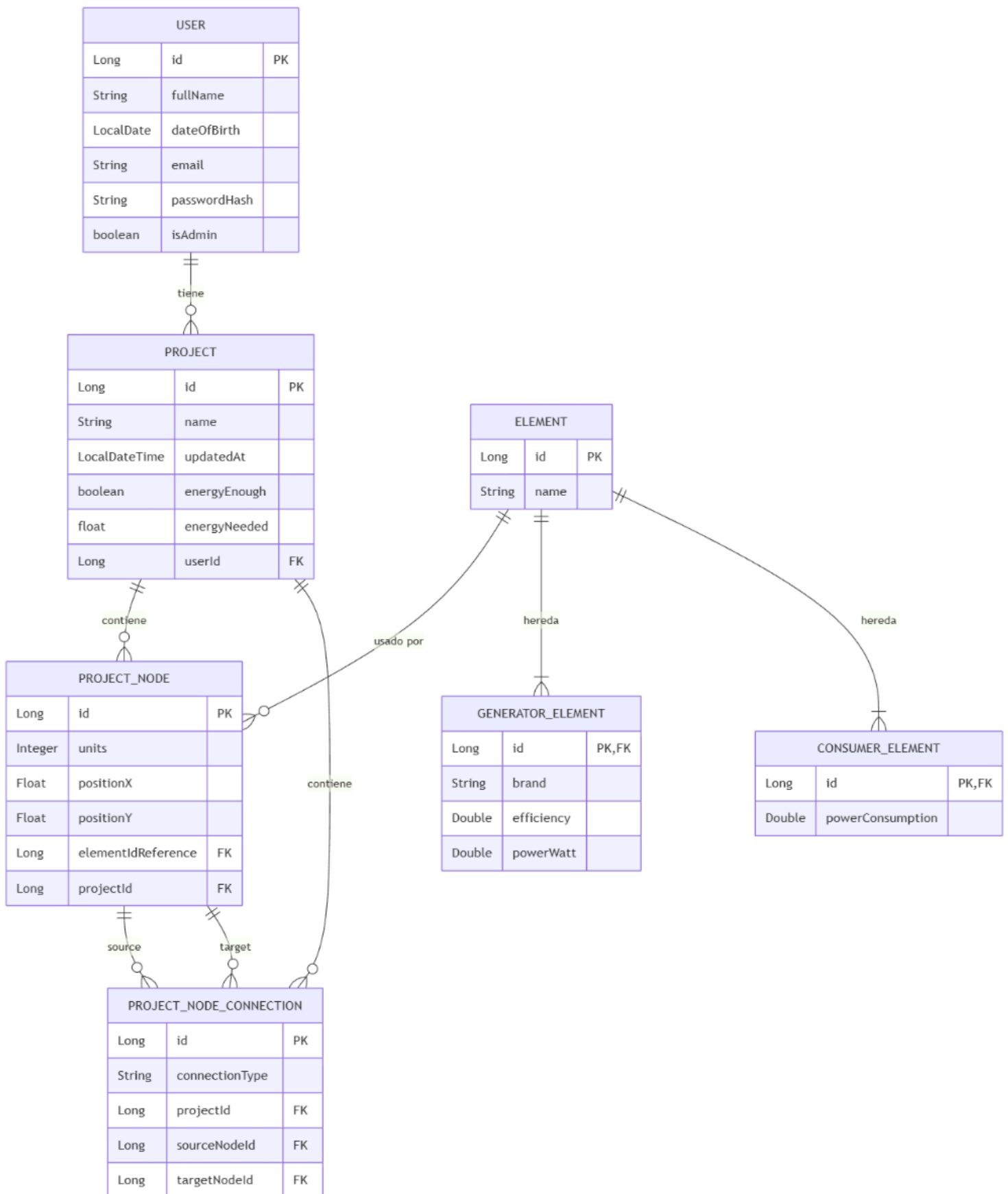
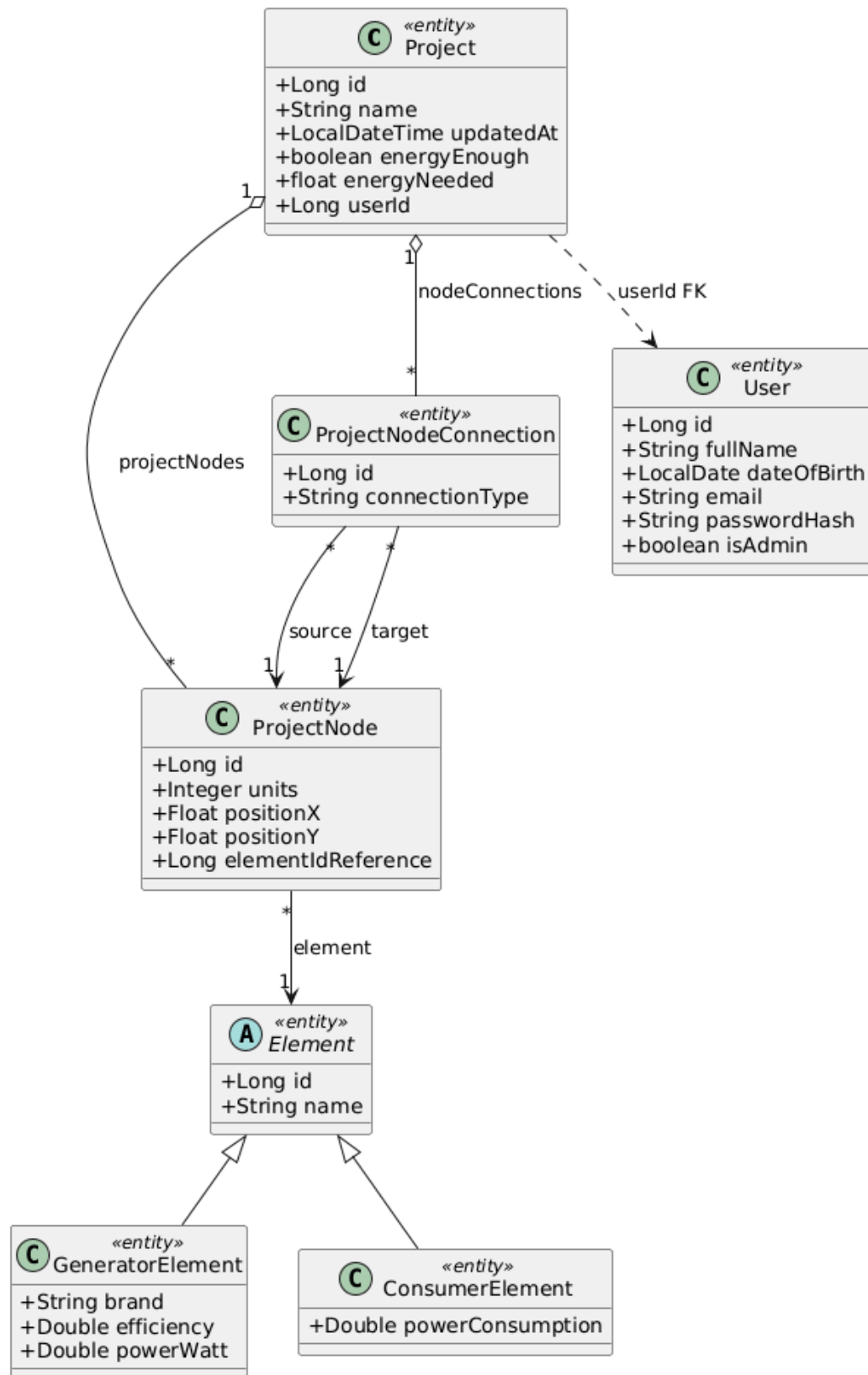


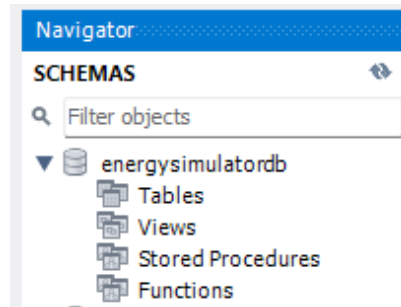
Diagrama UML



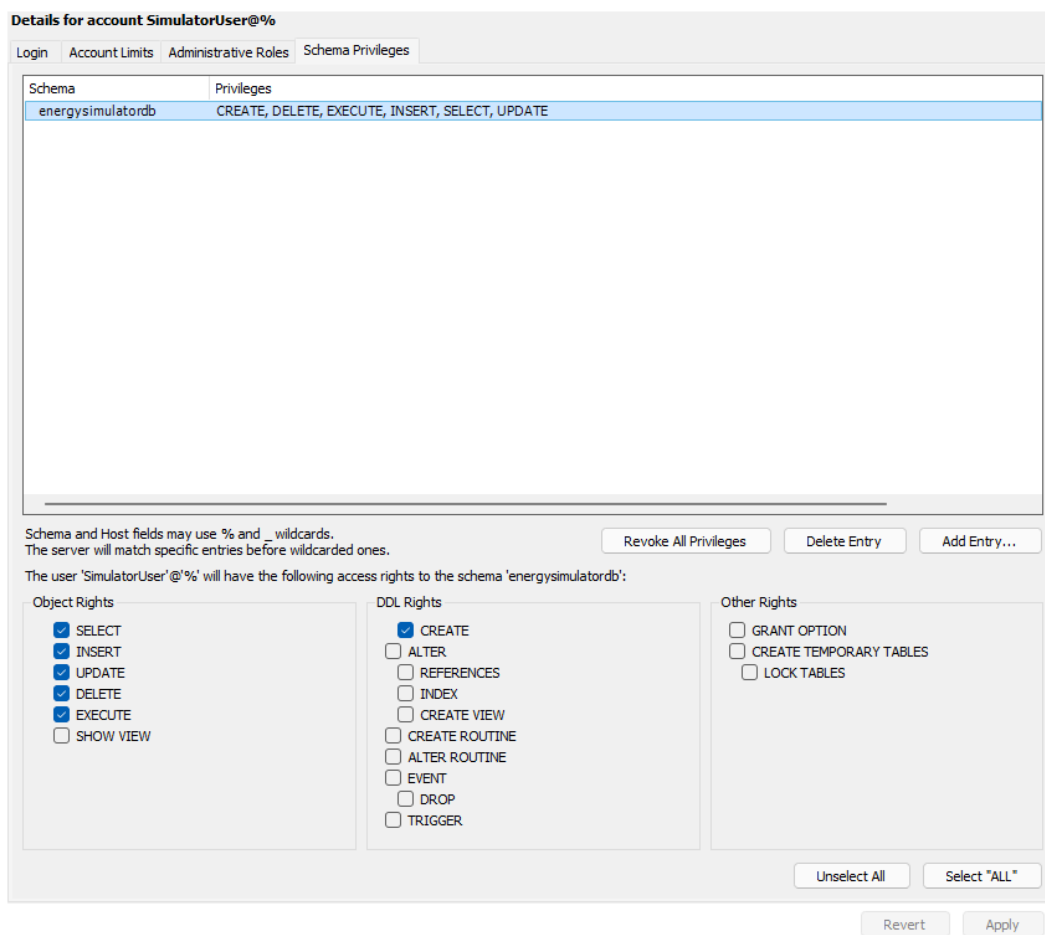
Configuraciones iniciales para el proyecto

para que el proyecto funcione correctamente, es necesario configurar lo siguiente:

1. En la máquina donde se vaya a ejecutar, debe tener instalado al menos, MySQL Server 8.X.X
2. En este, deberá estar creado la base de datos **energysimulatoredb**



3. Será necesario tener creado el usuario para el acceso a la base de datos
 - a. User: **SimulatorUser**
 - b. Password: **SimulatorUserPassword123**
 - c. Este se le asignará los permisos justos y necesarios para poder manejar la API



-
4. Se necesitará al menos **openJDK21 para poder compilar** y usar el proyecto (si ya está compilado, con JRE25 debería funcionar)
 - a. openJDK25 (distro de Microsoft) no funciona, el resto de JRE no se ha probado, por lo que no se garantiza su correcto funcionamiento.
 5. Para producción (si aplica): cambiar la IP/dominio de acceso a la API

Guía De Primer Contacto

A partir de aquí estaremos trabajando directamente en el código con el ORM Spring Boot. Para la persona que vea el proyecto backend por primera vez, se hace esta guía rápida a modo de introducción.

El proyecto principalmente consta de 4 carpetas y una clase iniciadora, que solo sirve para iniciar el proyecto.

Las carpetas son:

- **models:** Esta carpeta básicamente son las tablas que se crea en la base de datos. Se definen los atributos de la clase y contienen los constructores.
- **controllers:** Son las funciones de las tablas para acceder e interactuar con los datos
- **repositories:** Son las interfaces para que los controllers puedan interpretar las interacciones entre clases y tablas
- **config:** dentro están las configuraciones del proyecto, de momento solo hay un CorsConfig, donde se configuran las peticiones de la API

Para leer los scripts dentro de las carpetas, se recomienda verlos en el siguiente orden:

model>controllers>repositories.

Dentro de estas, la recomendación es seguir el flujo de la app, siendo:

Flujo Administrador

Crear un usuario administrador (**User**) >

Este crea los elementos necesarios (ya sea **ConsumerElement** o **GeneratorElement**) >

Terminamos

Flujo Usuario

Crear un usuario (**User**) > este crea un proyecto (**Project**) >

Entra en Project y añade elementos (en forma de nodos, **ProjectNode**) >

conecta los nodos, guardandolo en el proyecto (**ProjectNodeConnection**) >

Terminamos.

Pruebas

Se realizan las pruebas haciendo uso de postman, donde se comprobara que los endpoints que se han descrito en controllers hacen reciben correctamente la llamada y responden a las peticiones.

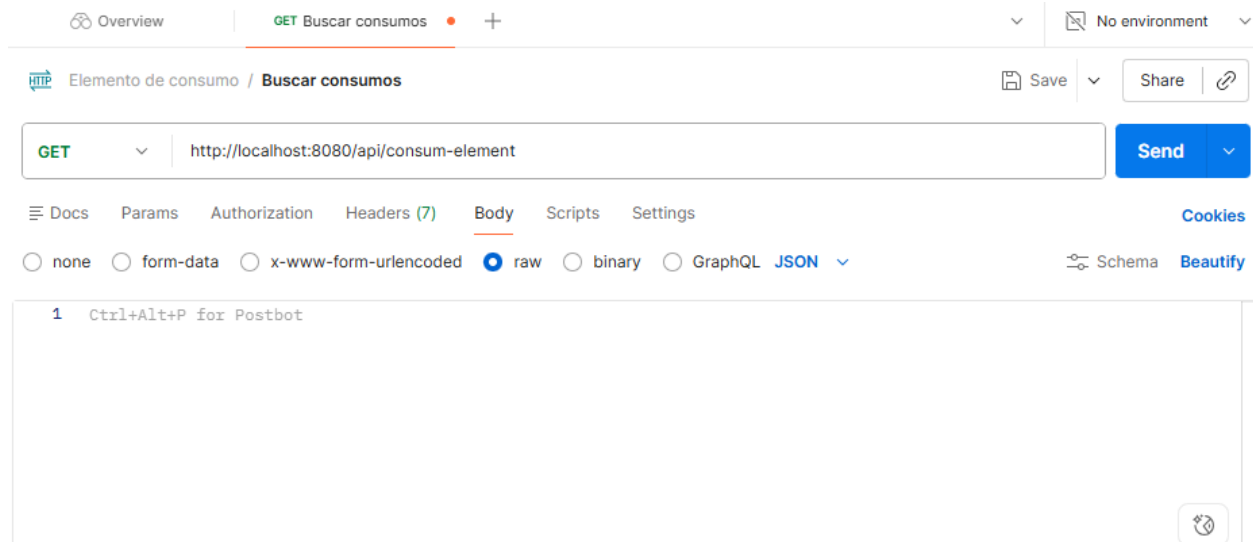
Se harán pruebas de POST, PUT, DELETE y GET de 3 de las clases que van a participar en el simulador



Se comprueba que responden correctamente a las peticiones.

Nos deberemos asegurar de que deberemos seleccionar correctamente el tipo de petición en la barra, GET, POST, PUT o DELETE que se encuentra al principio de la barra. Seguidamente se deberá poner la dirección en la que se hará la petición con el endpoint que nos interesa.

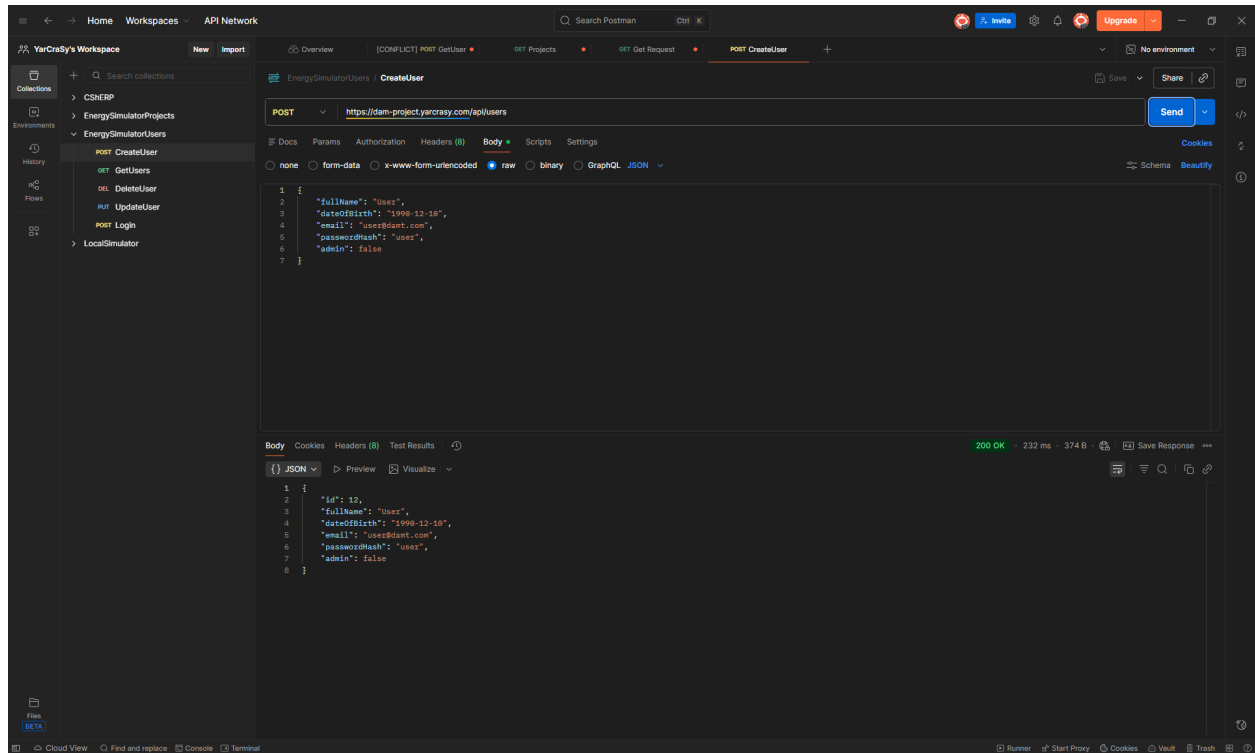
Nota importante: hay que seleccionar en body opción raw para que se lea y se envíe correctamente la información



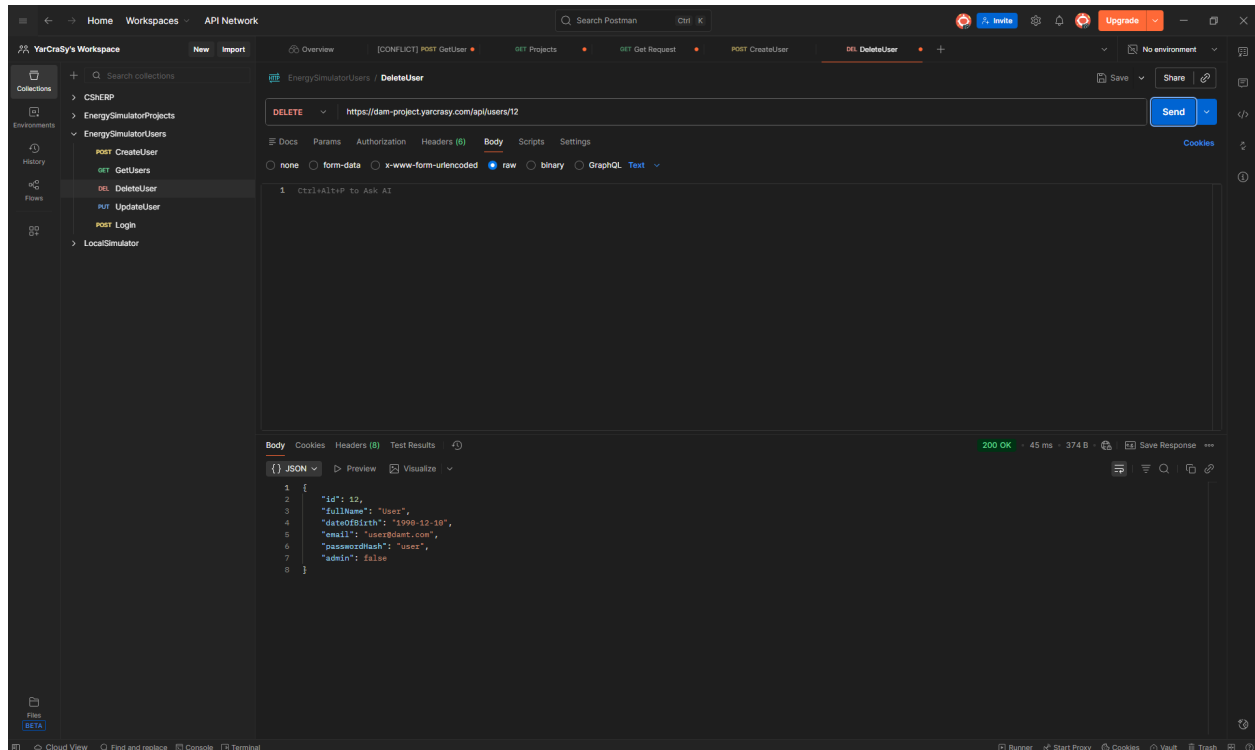
Prueba ejemplo de una petición POST para crear un usuario:

Deberemos introducir en formato JSON los valores que se han especificado en el constructor de la clase user.

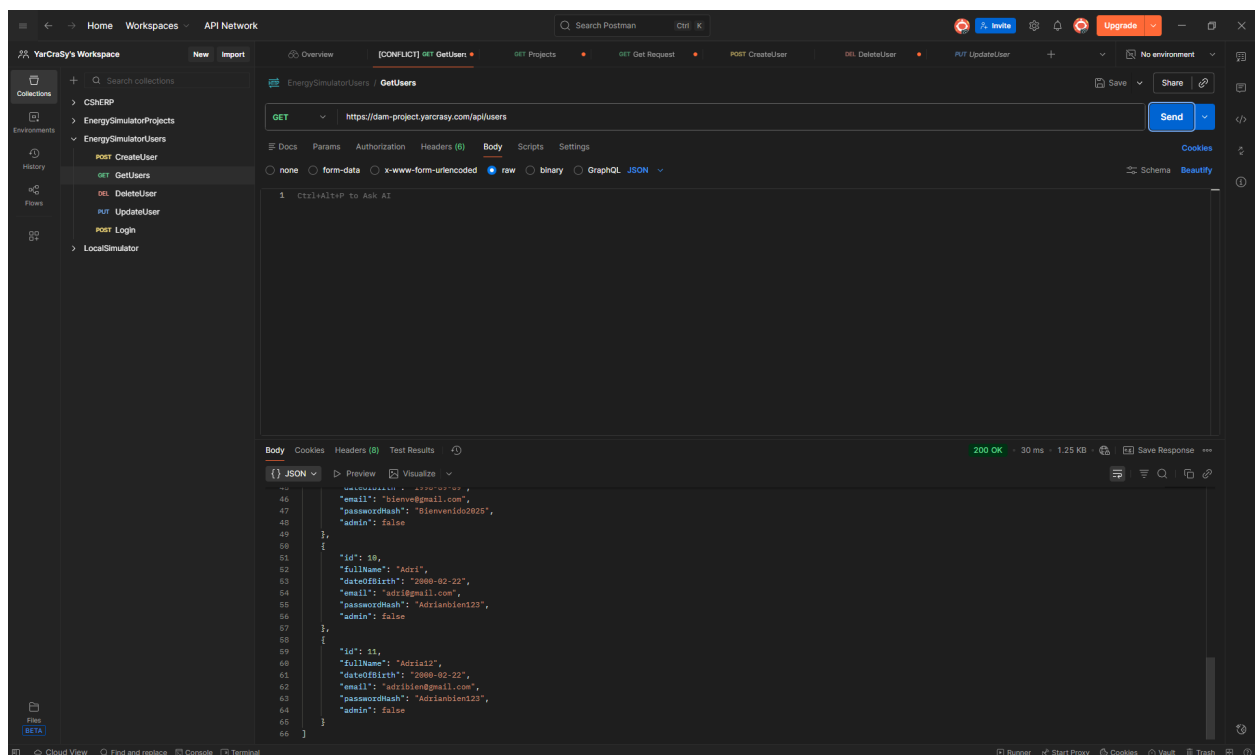
Nos podemos fijar en que se ejecuta correctamente porque el programa nos devuelve en la parte inferior de la pantalla los datos que hemos introducido, añadiendo el ID del usuario que se genera automáticamente, nosotros no tenemos que introducirla manualmente



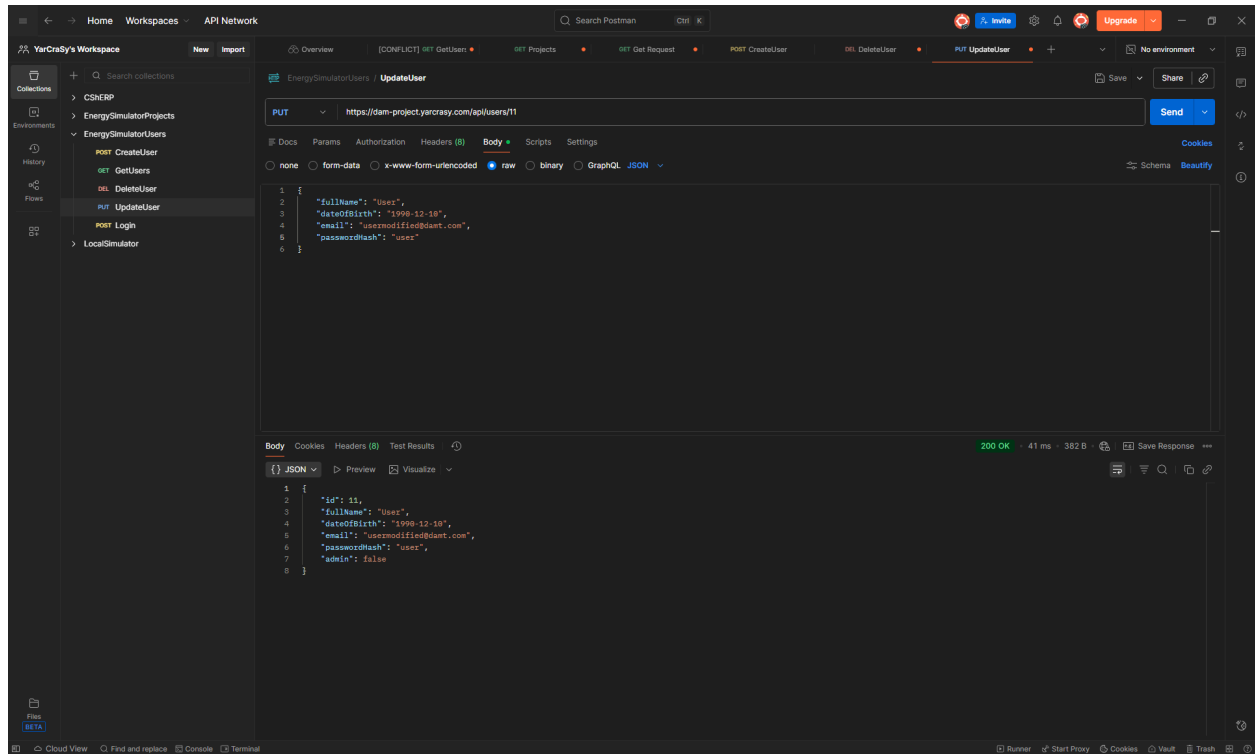
Ejemplo de DELETE de un usuario poniendo el ID del mismo cuando hacemos la llamada al endpoint, y observamos que el programa nos devuelve 200 ok, y en la parte inferior observamos los datos del usuario que se ha eliminado:



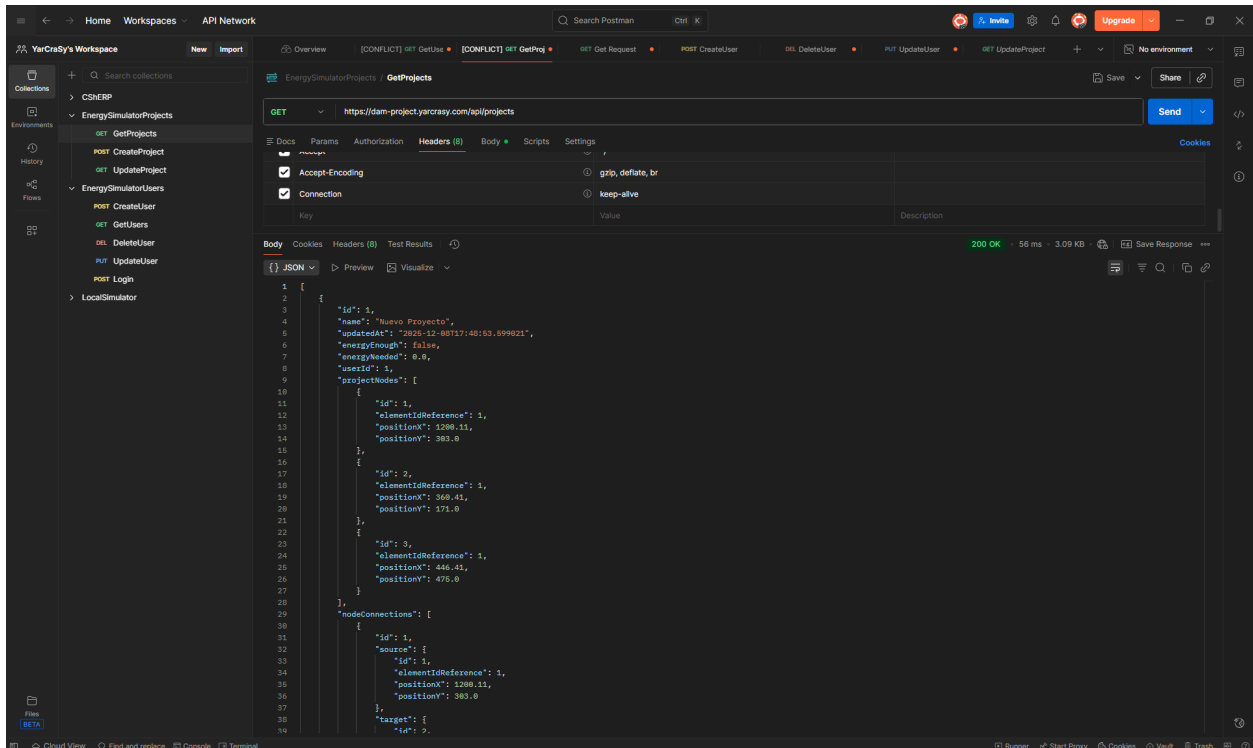
Haciendo uso del GET, obtenemos la lista de todos los usuarios que tenemos registrados en la base de datos actualmente, el programa nos devuelve 200ok y en la parte inferior se nos muestra en formato JSON todos los datos de los usuarios registrados:



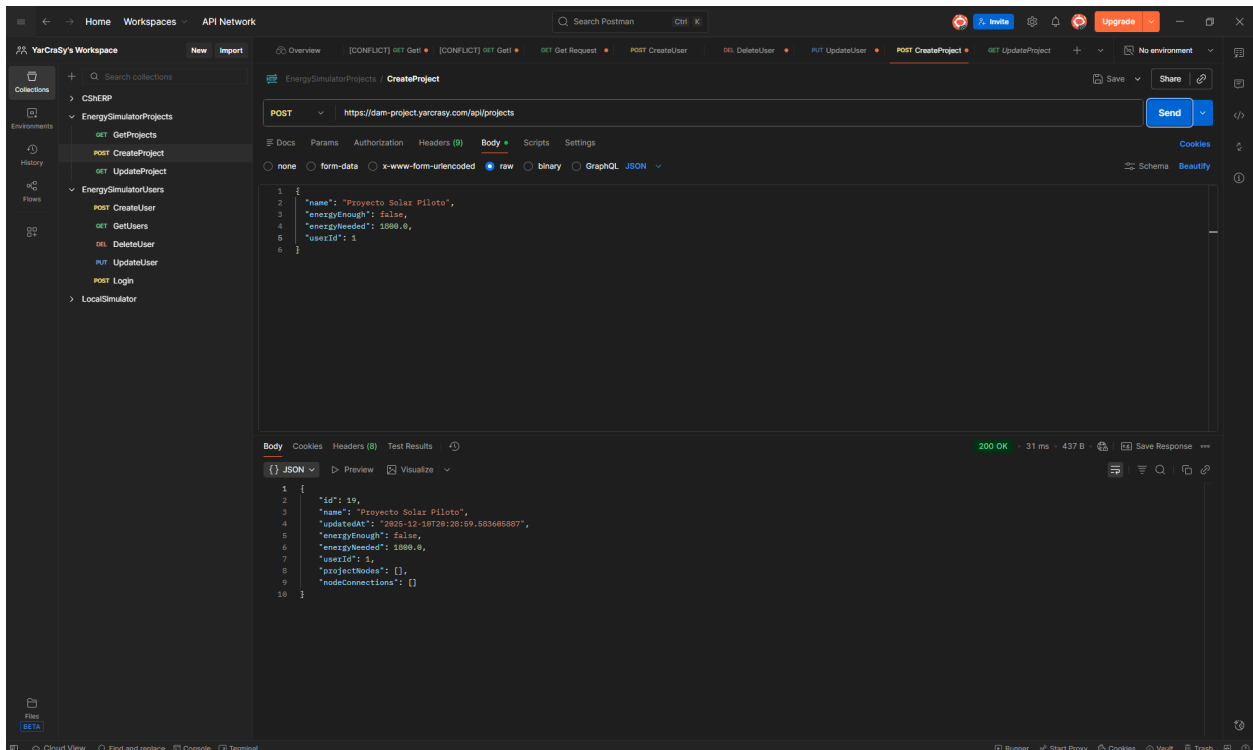
Al hacer PUT haciendo la llamada al endpoint de un usuario en concreto, debemos poner en el primer cuadrado en formato JSON todos los datos que usuario que queremos cambiar, con excepción del identificador, hacemos click en send y si todo va correcto se observa que en l parte inferior de la pantalla nos devuelve el usuario con los nuevos datos cambiados:



Se hacen las pruebas también en proyectos para obtener los proyectos que están registrados en la base de datos, haciendo un GET:



Igualmente, se pueden hacer POST para cerrar correctamente los proyectos como se muestra a continuación.



Problemas de conexión:

Cuando hay problemas con el acceso a la base de datos o alguna inconsistencia en los modelos o repositorios, nos podían aparecer:

- error 403: El servidor entendió la petición, pero no hay permisos para hacer lo que queremos.
- error 500: El servidor tuvo un fallo interno y no pudo procesar la petición. Error en el backend
- error 415: El servidor no acepta el formato en el que enviamos los datos

Despliegue (DEPRECADO)

DEBIDO AL AUMENTO DE COMPLEJIDAD DEL PROYECTO, SE DECIDE REESTRUCTURAR LA MÁQUINA DONDE SE HOSTEARÁ EL TODO EL PROYECTO.

NUEVA DOCUMENTACIÓN DEL PROCESO:  **Gestión del Proyecto**

Por necesidades de la entrega, necesitamos desplegar la Web, en consecuencia, también el backend. Por ello, tras revisar varias soluciones, concluimos que la mejor solución es manejar nosotros mismo el servidor. Por lo que se hosteará tanto el backend como el frontend desde una Raspberry Pi 5 limpio. Para ello, se usará Nginx, Certbot y Cloudflare.

Para no entrar mucho en detalle de las configuraciones que se hicieron en el sistema, solo se documentará la forma de desplegar.

Despliegue Web

Antes del despliegue, siempre compilamos el proyecto generando la carpeta dist

- `npm run build`

para luego desplegar la web, simplemente desde el PowerShell de Windows, se usará el comando (suponiendo que estamos en la carpeta del proyecto):

- `scp -P 14784 -r ./Web/dist/*
lingc@dam-project.yarcrazy.com:/home/lingc/dam-project/web`

se pedirá la contraseña para acceder al servidor, donde introducimos:

- `!Damelrincon2526`

En caso de que no cargue los cambios, accedemos a la rbp(Raspberry Pi) por ssh, usando el comando:

- `ssh dam-project.yarcrazy.com -p 14784`

donde se pedirá la contraseña:

- `!Damelrincon2526`

y reiniciamos el servicio Nginx

- `sudo systemctl reload nginx`

Despliegue Backend

Este caso es similar al despliegue web, pero teniendo que parar el servicio de Spring Boot que está corriendo en la rbp en segundo plano. Para ello:

accedemos a la rbp por ssh, usando el comando:

- `ssh lingc@dam-project.yarcrazy.com -p 14784`

donde se pedirá la contraseña:

-
- !Damelrincon2526

una vez dentro, usamos el siguiente comando para parar el servicio backend

- `sudo systemctl stop backend-api.service`

volvemos a windows donde abrimos otra terminal de PowerShell aparte y ejecutamos con la contraseña (suponiendo que estamos en la carpeta del proyecto)

- `scp -P 14784 ./backend-api/build/libs/backend-api-0.0.1.jar lingc@dam-project.yarcrazy.com:/home/lingc/dam-project/backend-api/`
- !Damelrincon2526

cuando termine de subir el archivo, volvemos a la rbp y re-activamos el servicio backend

- `sudo systemctl start backend-api.service`

esperamos a que arranque unos 5-10 segundos, y comprobamos que se arrancó con:

- `sudo lsof -i :8080`

si todo sale bien, debería mostrar algo similar a:

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
java	20629	lingc	19u	IPv6	75617	0t0	TCP	*:http-alt (LISTEN)

Conclusiones

A lo largo de la creación del presente proyecto se han podido implementar todos los conocimientos adquiridos en el curso anterior y en el presente. Ha servido para interiorizar conocimientos y tener la experiencia completa de desarrollo de aplicación ya que debe implementarse completamente.

Se ha comprobado que las bases y la organización del proyecto son aspectos fundamentales para el desarrollo de un programa desde 0. Es fundamental la comunicación y el reparto de tareas en el equipo.

El uso de las nuevas tecnologías como SpringBoot han facilitado el acceso a la base de datos y se ha ahorrado tiempo que se ha podido invertir en perfeccionar relaciones entre tablas.

Los mayores desafíos fueron formar las relaciones de N:M y la relación reflexiva de elemento, ya que un elemento puede estar conectado con otro elemento en el proyecto. Además, también ha habido que solventar problemas de bucles infinitos cuando se establecen las relaciones, por ejemplo, de listas de proyectos en los usuarios o lista de elementos en un proyecto.